

Case Recommender: A Flexible and Extensible Python Framework for Recommender Systems

Arthur da Costa¹, Eduardo Fressato¹, Fernando Neto¹, Marcelo Manzato¹ and Ricardo Campello^{2,1}

¹Institute of Mathematical and Computer Sciences - University of São Paulo, São Carlos, SP, Brazil

²School of Mathematical Physical Sciences - University of Newcastle, Newcastle, Australia

fortes@icmc.usp.br, eduardofressato@usp.br, fernando.sooares.aguiar@usp.br,
mmanzato@icmc.usp.br and ricardo.campello@newcastle.edu.au

ABSTRACT

This paper presents a polished open-source Python-based recommender framework named Case Recommender, which provides a rich set of components from which developers can construct and evaluate customized recommender systems. It implements well-known and state-of-the-art algorithms in rating prediction and item recommendation scenarios. The main advantage of the Case Recommender is the possibility to integrate clustering and ensemble algorithms with recommendation engines, easing the development of more accurate and efficient approaches.

CCS CONCEPTS

• Information systems → Recommender systems;

KEYWORDS

Recommender Systems, Framework, Python

1 INTRODUCTION

A number of frameworks for Recommender Systems (RS) have been proposed by the scientific community, involving different programming languages, such as Java, C#, Python, among others[2]. However, most of them lack an integrated environment containing clustering and ensemble approaches which are capable to improve recommendation accuracy. On the one hand, clustering may support developers to pre-process their data to optimize or extend recommender algorithms. On the other hand, ensemble approaches are efficient tools to combine different types of data in a personalized way. The features available in each considered recommender library are summarized in Table 1.

Case Recommender was developed to provide flexibility and extensibility in research environments, while maintaining high performance, providing a variety of recommendation and clustering algorithms, as well as functions for data manipulation [1]. Its primary concern is to maximize usefulness for research and education, instead of large-scale commercial operations. The framework is also designed to support a wide variety of recommendation approaches, including content-based, collaborative filtering and

hybrid approaches. Case Recommender is published in PyPi¹, a international repository of software for the Python programming language, as an MIT License, making it easy for third parties to contribute with additional implementations and features.

2 THE FRAMEWORK

Case Recommender is currently being rewritten to support optimized calculations using known Python scientific libraries. We also developed and improved classes that allow developers to manipulate files, predict and evaluate models, compute (dis)similarities between users or items, beside use metadata and external information. The framework is now implemented in *Python 3* and it addresses two common scenarios in Recommender Systems: rating prediction and item recommendation, using explicit, implicit or both types of feedback in several recommender strategies.

The important features while designing our framework were to enable the computation of recommendations in large-scale, easy creation and extension of algorithms for different types of filtering and scenarios. Another feature is to support sparse and large datasets in a way that there is as little as possible overhead for storing data and intermediate results. Developers can choose between using one of the available recommender algorithms, combining multiple recommendations using one of the available ensemble techniques, or develop their own algorithm using the *BaseRatingPrediction* or *BaseItemRecommendation* classes. Up to the current version, the algorithms available in the framework are shown at Table 2 and their references can be found in the header of the respective code.

Case Recommender contains a generic interface for recommender systems implementations, among them the collaborative and content-based filtering approaches such as neighborhood-based (NB) and matrix factorization (MF) models, which are already available for use. Additionally, two clustering algorithms are available: PaCo and KMedoids, in order to support recommendation engines. Also, more than 15 different already implemented (dis)similarity metrics can be easily applied to the recommender and clustering interfaces, like cosine and Pearson, using Scipy basic optimized functions². Moreover, it allows easy integration with different input domains like databases, text files or python libraries.

2.1 Recommender Engines

Case Recommender contains a recommender engine composed of several algorithms described in the literature, such as clustering, neighborhood and matrix factorization models. The framework

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

RecSys '18, October 2–7, 2018, Vancouver, BC, Canada

© 2018 Copyright held by the owner/author(s).

ACM ISBN 978-1-4503-5901-6/18/10.

<https://doi.org/10.1145/3240323.3241611>

¹<https://pypi.org/project/CaseRecommender/>

²<http://docs.scipy.org>

Table 1: Comparison of free/open source recommender system toolkits.

Toolkits	Case Recommender	EasyRec	Mahout	LensKit	RiVal	MyMediaLite	Crab
Version	Atualizar	1.0.4	0.13.0	2.2.1	0.2	3.11	0.1
Last update	Jun 2018	May 2016	May 2018	Nov 2015	Aug 2015	Dec 2015	Jan 2012
Actively Developed	✓	-	✓	✓	-	✓	-
License	MIT	GPL 3	Apache 2.0	LGPL 2	Apache 2.0	GPL 3	BSD
Language	Python	Java	Java	Java	Java	C#	Python
Scalable	✓	-	✓	✓	-	✓	✓
Rating Prediction	✓	✓	✓	✓	✓	✓	✓
Item Recommender	✓	-	-	✓	-	✓	✓
CF Techniques	✓	✓	✓	✓	✓	✓	✓
CBF Techniques	✓	-	-	-	-	✓	-
Ensemble Techniques	✓	-	-	-	-	-	-
Clustering Techniques	✓	-	-	-	-	-	-

Table 2: Case Recommender Implemented Algorithms.

Approach	Item Recommender	Rating Prediction
Neighborhood-Based	User KNN, User Attr KNN, Item KNN, Item Attr KNN, Content Based	User KNN, User Attr KNN, Item KNN, Item Attr KNN
Matrix Factorization	BPR MF	MF, Item-MFMS, SVD, SVD++, GSVD++, Item NSVD1, User NSVD1
Non-Personalized	Most Popular, Random	Most Popular, Random
Ensemble	BPR Learning, Tag Based, Average Based	-
Clustering-based	PaCo, Group-based	-

provides an assortment of classes and methods that may be plugged together and customized to create and evaluate an ideal recommender for a particular purpose. Our framework also provides an extensible platform for ensemble and clustering approaches which allow developers to combine different recommender algorithm using different input data. Such ensemble approaches, as they are integrated in the framework, allowing their evaluation to be fair comparisons against individual filtering approaches. Its flexibility allows developers to build architectures for better accuracy depending on the application scenario. Our implementation provides a clear interface, where the ensemble and clustering strategies may receive the results of previous probe runs, and the set of results from previous trained recommenders or other pre-processing techniques.

2.2 Validation and Evaluation Metrics

Case Recommender contains routines for recommendation evaluation, including measures such as RMSE and MAE for the rating prediction task. For the item recommendation task, the available metrics are $\text{prec}@N$, $\text{recall}@N$, $\text{MAP}@N$ and $\text{NDCG}@N$. For both scenarios, internal K -fold cross-validation and Shuffle Sample is supported to validate the experiments. The framework also contains the All But One protocol and statistical tests, such as Wilcoxon signed-rank test and the two-sided paired t-test, in order to compare and evaluate the results.

3 INSTRUCTIONS AND DEMONSTRATION

Case Recommender is hosted at Github and PyPi repositories and it is freely available for developers and the scientific community to use, test and contribute under MIT license. To install Case Recommender on Mac OS X, Windows or Linux, the following command will work: *pip install caserecommender*.

Case Recommender comes with instructions that allow users to train and evaluate all available recommenders on data provided in text files. When a new recommender is added into the framework, it is automatically detected, so developers do not have to manually add new features into the programs, only use the command *import* in Python. The main instructions and demonstration of use are available at [Github](#)³ and [YouTube](#)⁴, respectively.

4 FINAL REMARKS

The goal of Case Recommender is to integrate and facilitate the experiments and development of new recommender techniques for different domains. Our framework contains a recommender engine, that contains content-based, collaborative and hybrid filtering approaches for rating prediction and item recommendation scenarios. In addition, the framework contains ensemble and clustering algorithms, validation and evaluation metrics to improve and measure the quality of the recommendation.

Future releases are planned which will include more features (parallel processing techniques and new algorithms) and an evaluation tool with several plots and graphs to help developers to better understand the behavior of their recommender algorithms.

ACKNOWLEDGMENTS

We would like to acknowledge FAPESP (2016/20280-6) CAPES and CNPq for the financial support.

REFERENCES

- [1] Arthur F da Costa and Marcelo G Manzato. 2016. Case Recommender: A Recommender Framework. In *Workshop of Tools and Applications (WEA)*, 16, 2016. XXII WebMedia. , 2016. v. 2.
- [2] Zeno Gantner, Steffen Rendle, Christoph Freudenthaler, and Lars Schmidt-Thieme. 2011. MyMediaLite: A Free Recommender System Library. In *Proceedings of the Fifth ACM Conference on Recommender Systems (RecSys '11)*. ACM, New York, NY, USA, 305–308. <https://doi.org/10.1145/2043932.2043989>

³<https://github.com/caserec2018/caserecommender>

⁴www.youtube.com/watch?v=NHCQQD3ZSNA